

Writing Test Cases That Actually Catch Salesforce Bugs

In Salesforce testing, writing more test cases does not necessarily mean better testing. A test suite can contain hundreds of test cases and still miss critical bugs if those cases only validate the “happy path.”

Effective Salesforce testing is about thinking beyond “**Does the feature work?**” and asking “**What could go wrong, and under what conditions?**”

This article explains how to write Salesforce test cases that are practical, risk-focused, and capable of finding real-world defects.

1. Start With the Business Requirement

Before writing a test case, understand what the feature is actually supposed to achieve.

For example, suppose the requirement is:

When a customer sends a STOP keyword, the customer should be unsubscribed and users should not be able to send further messages.

A weak test case might be:

Test: Send STOP and verify the customer is unsubscribed.

A stronger approach considers the complete business flow:

- What happens when STOP is received?
- Is the correct record created?
- Which user should be prevented from sending?
- Does the restriction apply to one sender number or all sender numbers?
- What happens if the user tries to send an SMS?
- What happens through another supported channel?
- Does the same behavior work for System Admin and Community users?
- What happens if the customer opts in again?

The goal is to convert the requirement into **business scenarios**, not just UI steps.

2. Cover Positive, Negative, and Boundary Scenarios

A good Salesforce test case should not only verify successful behavior.

Consider a field that accepts a maximum of 255 characters.

Instead of testing only:

Enter 100 characters and save.

Test:

- Minimum valid value
- Typical valid value
- Maximum valid value
- Maximum + 1 character
- Blank value
- Spaces only
- Special characters
- Unicode characters
- Copy/paste values
- Invalid data format

This is where many defects are discovered.

Think in terms of:

Valid → Invalid → Boundary → Unexpected

That simple pattern can significantly improve test coverage.

3. Test Salesforce Permissions and Profiles

Salesforce behavior can change depending on the user's profile, permission sets, licenses, sharing settings, and object permissions.

For important functionality, do not test only with System Administrator.

For example, consider:

- System Administrator
- Standard User
- Community/Experience Cloud User
- Users with restricted object permissions
- Users with different permission sets
- Users without access to specific records

Verify:

- Can the user see the feature?
- Can the user create records?
- Can the user edit records?
- Can the user delete records?

- Can the user access related records?
- Does the correct error message appear when access is denied?

A feature that works perfectly for an administrator may still be broken for the actual end user.

4. Test Record-Level Security

Salesforce security is not limited to object permissions.

You should also consider:

- Organization-Wide Defaults
- Role hierarchy
- Sharing rules
- Manual sharing
- Permission sets
- Record ownership
- Field-Level Security
- Profile permissions

For example, if User A owns a record and User B does not have access to it, verify that User B cannot access the record through:

- Direct record URL
- Search
- Related lists
- Reports
- List views
- Global search
- Custom Lightning components
- Mobile application

A common mistake is testing access only through the standard UI.

5. Test Both Desktop and Mobile Experiences

If a Salesforce feature is integrated with a mobile application, the same business scenario should be validated on both platforms.

For example:

Desktop:

Customer sends STOP → unsubscribe record created → user attempts to send message → correct error displayed.

Mobile:

Customer sends STOP → unsubscribe record created → user attempts to send message → same restriction and expected error displayed.

The important question is not whether the UI looks identical.

The question is:

Does the business behavior remain consistent across platforms?

Differences between desktop and mobile implementations are a common source of defects.

6. Test Integrations, Not Just Salesforce

Salesforce applications frequently depend on external systems and packages.

For example:

Salesforce → CTI → Telephony Provider → Customer

or

Customer → SMS Provider → Salesforce → Mobile App

Testing only Salesforce UI is not enough.

For an integration test, verify:

1. Request is generated correctly.
2. Request reaches the external system.
3. External system processes the request.
4. Salesforce receives the response.
5. Correct Salesforce record is created or updated.
6. UI reflects the latest status.
7. Errors are handled correctly.

Also test failures such as:

- API timeout
- Invalid authentication
- Expired token
- Invalid response
- Duplicate response
- Network interruption
- Service unavailable
- Partial transaction failure

Integration failures often expose bugs that normal functional testing misses.

7. Test State Changes

Many Salesforce bugs occur when records move from one state to another.

For example:

New → Active → Inactive

or:

Subscribed → Unsubscribed → Subscribed

Don't test only the initial state.

Test every meaningful transition.

For example:

Current State	Action	Expected State
Subscribed	STOP received	Unsubscribed
Unsubscribed	Send message	Message blocked
Unsubscribed	Opt-in	Subscribed
Subscribed	Send message	Message sent

Then test invalid transitions as well.

This approach is particularly useful for compliance, subscription, call, messaging, and approval-related functionality.

8. Test Data Combinations

Salesforce defects often appear only when multiple configuration values interact.

For example:

- Compliance Mechanism = Record Wise
- Sub Type = Sender Number Wise
- Sender Number = P1
- Incoming Number = Customer A

Now change one variable at a time:

- Sender Number P2
- Different customer
- Different user
- Different channel
- Different Salesforce record
- Different compliance configuration

This helps identify whether the functionality is actually working according to the configuration.

A useful technique is to create a **test matrix**.

Example

User	Sender	Customer	Channel	Expected
Admin	P1	C1	SMS	Block
Admin	P2	C1	SMS	Allow
Community User	P1	C1	SMS	Block
Admin	P1	C2	SMS	Allow

This is much more powerful than testing a single combination.

9. Verify Error Messages

Error handling deserves its own test cases.

Don't simply verify that an action fails.

Verify:

- Correct error message
- Correct error location
- Message is understandable
- No misleading message appears
- No technical/internal information is exposed
- Same business rule produces consistent messaging across platforms

For example:

Customer has opted out from all communications from your organization.

If the desktop application displays the expected error but the mobile application allows the message to be sent, the problem is not merely a UI issue—it is a **business-rule consistency defect**.

10. Test Permissions After Configuration Changes

Salesforce administrators frequently change configurations.

Therefore, test scenarios such as:

- Permission added
- Permission removed
- Permission changed
- Package upgraded
- Package downgraded
- Feature enabled
- Feature disabled
- Configuration changed
- User profile changed

For managed packages, regression testing after an upgrade is particularly important.

A previously working feature may break because of:

- Changed metadata
- Dependency changes
- Permission changes
- API changes
- New validation rules
- Updated Lightning components
- Package configuration changes

11. Test Bulk Data

Salesforce is designed to handle large volumes of records, so testing a single record is not enough.

Where applicable, test:

- 1 record
- Multiple records
- Large data volume
- Bulk update
- Bulk delete
- Bulk import
- Duplicate records
- Records with missing data

For example, if a trigger or Flow updates related records, verify that the process works when 1 record is processed as well as when many records are processed together.

This is especially important for:

- Apex triggers
- Flows
- Batch processes
- Integrations
- Data migrations

12. Test Automation and Flows

Salesforce automation can create unexpected side effects.

If a record update triggers:

Flow → Field Update → Another Flow → Task Creation

you should verify the complete chain.

Test:

- Automation runs when expected.
- Automation does not run when it shouldn't.
- Correct records are updated.
- No duplicate records are created.
- No recursive behavior occurs.
- Errors are handled correctly.
- Existing functionality is not affected.

Always ask:

What else changes because I performed this action?

That question can uncover hidden defects.

13. Write Test Cases With Clear Expected Results

Avoid vague expected results such as:

“System should work correctly.”

Instead, describe the exact expected behavior.

Weak

Verify message functionality.

Strong

When the user sends a message from sender number P2 to customer C1, the message should be sent successfully, a message record should be created in Salesforce, and the message status should be updated to Sent.

A strong expected result should tell another tester exactly what to verify.

14. Include Regression Scenarios

Whenever a new feature is introduced, identify what existing functionality could be affected.

For example, if a new compliance mechanism is introduced, regression testing should cover:

- Existing SMS functionality
- MMS
- WhatsApp
- Incoming messages
- Outgoing messages
- Templates
- Mobile application
- Desktop application
- User permissions
- Existing compliance configurations

Feature testing answers:

Does the new feature work?

Regression testing answers:

Did the new feature break anything that already worked?

Both are necessary.

15. Use Risk-Based Testing

Not every functionality deserves the same testing effort.

Prioritize areas that can cause:

High Business Impact + High Probability of Failure

For Salesforce applications, these commonly include:

- Authentication

- Permissions
- Data security
- Customer communication
- Payments
- Integrations
- Data creation/update
- Compliance
- Automation
- Package upgrades

Spend more time testing areas where a defect could result in data loss, security problems, customer impact, or business disruption.

16. Think Like a Developer—and Like a User

The best QA testers switch between two perspectives.

Think like a developer

Ask:

- What happens if the API fails?
- What if a required field is null?
- What if the record doesn't exist?
- What if the same request comes twice?
- What if two processes update the record simultaneously?
- What happens after a package upgrade?

Think like a user

Ask:

- Is the feature easy to understand?
- Is the error message clear?
- Can I complete my task?
- What happens if I make a mistake?
- Does the feature behave consistently?
- Can I access something I shouldn't?

Combining both perspectives produces much stronger test coverage.

17. A Practical Test Case Formula

A simple formula can help when creating Salesforce test cases:

Requirement → User → Data → Configuration → Action → Expected Result → Negative Scenario → Boundary → Integration → Regression

For example:

Requirement: Customer must not receive messages after opting out.

Then consider:

- Which user?
- Which customer?
- Which sender number?
- Which compliance configuration?
- Which channel?
- What happens when STOP is received?
- What record should be created?
- What happens when the user sends a message?
- What happens through another sender number?
- What happens through another channel?
- What happens after opt-in?
- Does the behavior remain consistent on mobile and desktop?

This turns a simple requirement into meaningful test coverage.

Final Thoughts

Writing effective Salesforce test cases is not about creating the largest possible test suite.

It is about creating **smart test coverage**.

A strong Salesforce QA tester doesn't stop at:

"The expected result is working."

They ask:

"What happens if the user has different permissions?"

"What happens with different data?"

"What happens when the configuration changes?"

"What happens when the integration fails?"

"What happens on mobile?"

"What happens after an upgrade?"

"What happens when the user does something unexpected?"

That mindset is what turns test cases from documentation into a powerful tool for finding real Salesforce bugs.

The goal of a good test case is not to prove that the software works. The goal is to discover where it doesn't.